

Task 3: Model Validation, Overfitting Control & Hyperparameter Tuning

Project: California House Price Prediction System

1. Introduction and Objective

In machine learning, achieving high accuracy on training data is a trivial task if the model lacks the ability to generalize to unseen data. The primary objective of this task is to transition from basic model training to professional model validation and optimization. Using the California Housing Dataset, this report details the process of detecting model overfitting, establishing a reliable performance baseline using cross-validation, and conducting systematic hyperparameter tuning to select an optimized, production-ready regression model.

2. Comprehensive Overfitting Analysis

Overfitting occurs when a machine learning model learns not only the underlying patterns of the training data but also its statistical noise and outliers. This results in a model with low bias but exceptionally high variance, making it highly unreliable when exposed to new data.

To demonstrate this phenomenon, a baseline, unconstrained `DecisionTreeRegressor` was trained on the scaled features. Tree-based models are particularly prone to overfitting because, without defined stopping criteria, they will continue to split nodes until every leaf is pure (containing only one sample).

The experimental results strictly confirmed this theoretical behavior:

- **Training Performance:** The model achieved a Training Root Mean Squared Error (RMSE) of $3.017e-16$, which is practically zero. This indicates that the tree perfectly memorized the entire training dataset.
- **Testing Performance:** When evaluated on the unseen 20% test split, the model returned a significantly higher Test RMSE of 0.702.

The massive discrepancy between the near-perfect training score and the poor testing score is a textbook example of severe overfitting. The baseline model is highly unstable and completely unsuitable for deployment in its current state.

3. Cross-Validation Methodology

Relying on a single train-test split can yield misleading performance metrics, as the results are highly dependent on the random initialization of the split. The data in the test set might happen to be exceptionally easy or exceptionally difficult for the model to predict.

To establish a trustworthy and realistic performance metric, a K-Fold Cross-Validation strategy was implemented. The dataset was divided into 5 equal folds (`cv=5`). The model was trained on

4 folds and tested on the 1 remaining fold, iterating this process 5 times so that every data point served as testing data exactly once.

Using the negative root mean squared error scoring parameter, the cross-validation process yielded an average CV RMSE of 0.896. This score is noticeably worse than the single test split score (0.702), proving that the single split was overly optimistic. The CV RMSE of 0.896 represents the true baseline generalization capability of the unconstrained tree, validating the necessity of cross-validation for professional ML evaluation.

4. Hyperparameter Tuning Approach

To cure the overfitting problem, it is necessary to constrain the decision tree's complexity, thereby managing the bias-variance tradeoff. Instead of manually guessing the best constraints, an automated and systematic search was conducted using [GridSearchCV](#).

A parameter grid was defined to test multiple structural limits:

- **max_depth [3, 5, 7, 10]:** This hyperparameter dictates how deep the tree is allowed to grow. Limiting depth prevents the model from learning hyper-specific noise.
- **min_samples_split [2, 5, 10]:** This hyperparameter requires a node to have a minimum number of samples before it is allowed to split further, preventing the creation of highly specific branches.

The [GridSearchCV](#) algorithm evaluated every possible combination of these parameters utilizing 5-fold cross-validation to ensure the tuning process itself did not overfit to a specific test set.

The automated tuning process successfully identified the optimal parameters for this dataset:

- **Best max_depth:** 10
- **Best min_samples_split:** 10

These parameters strike the optimal balance. A depth of 10 is deep enough to capture the complex, non-linear relationships in the California housing data (preventing underfitting), while the minimum split requirement of 10 prevents the tree from isolating individual noisy data points (preventing overfitting).

5. Final Model Selection and Justification

Following the tuning process, the optimized Decision Tree was evaluated against the testing data and compared with the baseline models developed in Task 2 (Linear Regression and Ridge Regression).

The performance metrics clearly justify the selection of the Tuned Decision Tree as the final production model:

- **Linear Regression:** RMSE = 0.745, R2 Score = 0.575
- **Ridge Regression:** RMSE = 0.745, R2 Score = 0.575
- **Tuned Decision Tree:** RMSE = 0.645, R2 Score = 0.682

Justification:

1. **Superior Predictive Accuracy:** The Tuned Decision Tree outperforms both linear models, reducing the prediction error (RMSE) from 0.745 to 0.645 and increasing the explained variance (R2 Score) from roughly 57% to over 68%.
2. **Capturing Non-Linearity:** The linear models underfit the data because house prices do not scale perfectly linearly with features like population or geographic coordinates. The Decision Tree easily maps these non-linear geographical and demographic boundaries.
3. **Controlled Overfitting:** Unlike the initial baseline tree that memorized the data, the tuned model successfully generalizes. The hyperparameter tuning applied strict structural limitations (`max_depth` and `min_samples_split`), resulting in a reliable model that balances complexity and generalizability.

In conclusion, the Tuned Decision Tree represents a highly optimized, cross-validated system that satisfies industry standards for machine learning deployment. The rigorous validation and tuning workflows have transformed a volatile, overfitting algorithm into a robust, high-performing predictive model.